Dead Lock :-

In multiprogramming Environment we have several process and several no of Resources.

what is deadlock

two or more Process are waiting on some Res avilability. which will never be avilable as it also busy with some other Process. the Process are said to be in deadlock.

→ DL is big Present in the Process/thread synchronization methord

→ In DL Process never finish executing. and sys. Resource are tried up, preventing other job from starting.

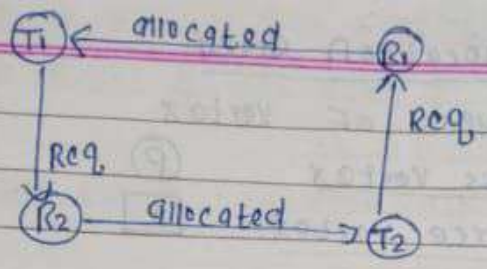
→ Example of Resource :- Memory space, CPU cycl files, locks, sockets, IO devices etc.

⊙ How a Process/thread Utilize a Resources :-

① Request: Request the R
if R is Free lock it else wait till it is avilable.

② Use

③ Release :- Relese Resource instance and make it avilable for other Process.



→ Deadlock Necessary condition :-

all 4 condition simultanusly true than dead occur.

(a) mutual exclusion :-

(only one Process at a time can use the resource.)

If another Process Requests that Resource the Requested Process must wait until the Resource has been Released.

(b) Hold and wait

A Process must be holding at least one Resources and wait to acquire addition Resource that are currently held by other Process.

(c) No-Prempton :- Resource must be Voluntarily Released by the Process after Completion of execution.

(P) ← [R]

(d) circular wait :-

A set {P0, P1, P2 - - Pn} of waiting Process must be exist such that P0 is waiting for Resource held by P1, P1 is waiting for Resource held by P2 and so on.

⇒ methord for handling deadlock :-

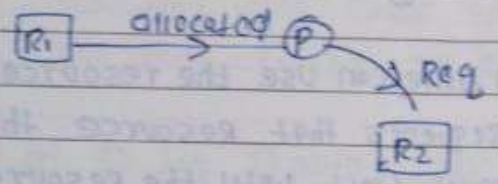
→ Resource allocation Graph

① two types of vertex

- Process vertex (P)
- Resource vertex [R]

② edges ① Assign

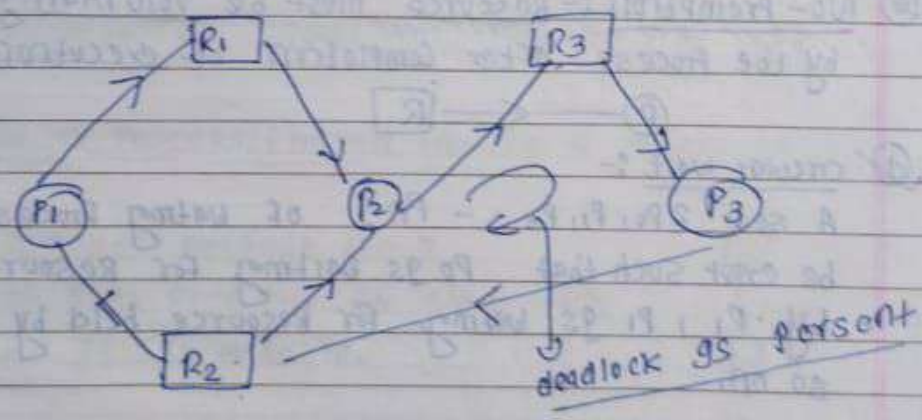
② Request Edge



③ multiple instances :- [R..]

suppose core i3 one CPU have 3 core CPU core CPU called instance of CPU.

single instance :- [R]

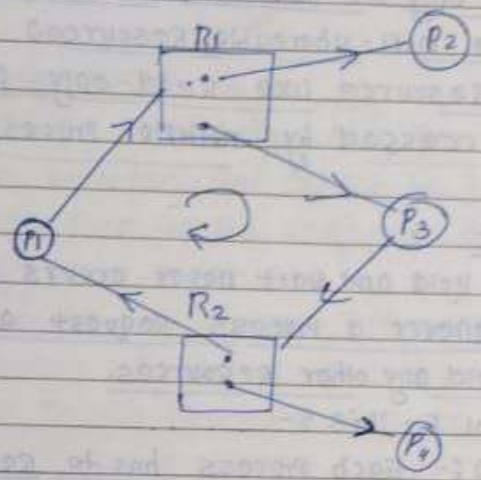


[R4 :]

important

(If there is a cycle then deadlock (possible) if no cycle then no deadlock.)

→ why we use word if ~~[illegible]~~ cycle than may be deadlock



here cycle but no deadlock
later P4 edge is remove (P4 process relese the R2 resource) now this instance is free for P3 process. and when P2 process relese R1 resources than now this resource instance is free for P1. so here is no deadlock

⊙ methord to handle deadlock

① Prevent or avoid deadlocks :- ensuring that the system will never enter a deadlock state.

② allow system to go in DL :- one algorithm detect dead lock and other recover.

③ ostrich algorithm :- Deadlock ignorance. here OS not handle deadlock. Application Programmer handle deadlock by writing Programe

⑥ DL Prevention :-

by ensuring at least one of the necessary condition can not hold.

① mutual exclusion

→ use locks only for non-sharable Resource.

→ (Printer are non-shareable Resource.)

→ (sharable Resource like Read-only-File can be accessed by multiple process/thread.)

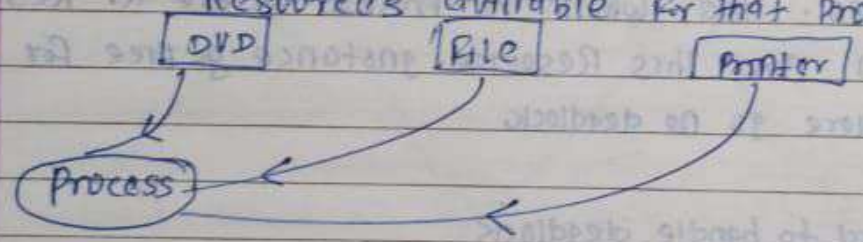
② Hold & wait

to ensure Hold and wait never occurs in the system. whenever a process request a Resources it doesn't hold any other resources.

two Protocol for this :-

① Protocol (A) :- Each process has to request and be allocated all it's Resources before it's execution.

Ex :- a Process start execution when All Resources available for that process.

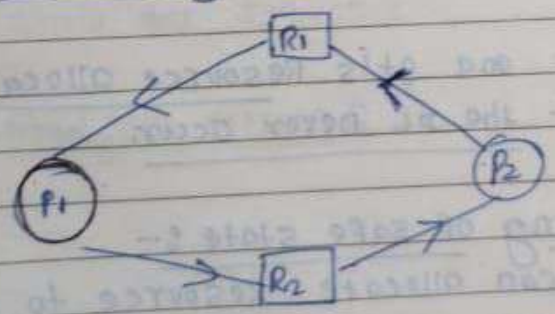


copy the data from DVD to File and now Printer print. so before execution start. we Request DVD, File, Printer. when this is available than start execution.

(b) Protocol (B) :- allow a process to Request resource only when it has none.
(~~it~~ Process can Request any additional resources after it must have released all the resources that is currently allocated.)

(c) No preemption :-
(i) if a Process is holding some Resources and Request another Resources that cannot be immediatly allocated to it then all the Resources the Process currently holding are Preempted.
the Process will Restart only when it Regain old Resources, as well as new one that it's Requesting
(ii) if a Process Requests some Resources. First check weather they are avilable. if yes we allocate them. if not, we check weather they are allocated to some other Process that is waiting for additional resources. if so Preempt the desired Resource from waiting Process and allocate them to Requesting Process.

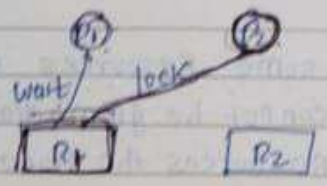
(d) Circular waiting :-



P1 Process require both R1 and R2 and P2 Process also require both R1 and R2

Proper ordering of Resource allocation

P_1 and P_2 both require R_1. and R_1 (locking on those resource should be like) both try to lock R_1 then R_2. by this way whichever process first lock R_1 will Got R_2.



[two or more than two Process request for the Resource. whichever Process not lock the Resource. will not request for another Resource.]

LEC-

→ Deadlock Avoidance

deadlock Detection and Recovery

the meaning of deadlock Avoidance is we know current state of system. current state means

① no of Process in the system.
② Need of Resources of Each Process.
③ Currently allocated amount of Resource to Each Process.
④ max amount of Each Resources.

ⓐ schedule Process and it's Resource allocation in such away that the DL never occur.

ⓑ what is meaning of safe state :-
if the system can allocate Resource to each Process in some order. then we avoid dead lock.

safe state means safe sequence

For Example $P_1 \to P_2 \to P_3 \to P_4$ than deadlock not occur. but $P_1 \to P_3 \to P_2 \to P_4$ than deadlock occur.

(c) For understanding purpus

	allocated			max need			avilable			Remaning need			
P	A	B	C	A	B	C	A	B	C	A	B	C	
P_1	0	1	0	7	5	3	3	3	2	~~7~~	~~4~~	~~3~~	P_1
P_2	2	0	0	3	2	2	5	3	2	~~1~~	~~2~~	~~2~~	P_2
P_3	3	0	2	9	0	2	7	4	3	~~6~~	~~0~~	~~0~~	P_3
P_4	2	1	1	4	2	2	7	4	5	~~2~~	~~1~~	~~1~~	P_4
P_5	0	0	2	5	3	3	7	5	5	~~5~~	~~3~~	~~1~~	P_5
total already	7	2	5				10	5	7				

~~कुछ तरह से Process~~

किस तरह से Process को resource allocate करना कि मैं safe state में ला सूँ

suppose A के 10 instance हैं, B के 5 और C के 7

P_2 P_4 P_5 P_1 P_3

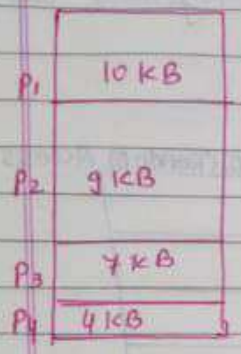
this is safe state here is no dead lock if we follow this sequence

(d) Dead lock detection

मेहनत करना पड़ता है कुछ पाने के लिए आराम से करो
अच्छे से करो

Page No.:
Date: / /

What is paging (non contiguous memory allocation)

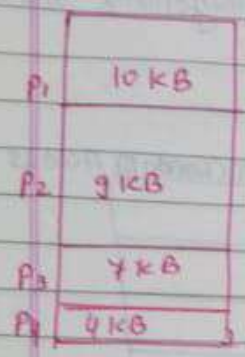


- memory management OS :-
- Logical Address (virtual address)
  - a address generated by CPU.
  - ② User can indirect access the the physical address through logical address.
  - ③ Logical address does not exist physically.
  - ④ Range : 0 to max.

- Physical address :-
  - ① User can never access physical address of the programe.
  - ② a physical address can be accessed by a user indirectly but not directly.
  - ③ it's computed by (MMU) (memory management unit)
  - ④ Range: (R+0) to (R+ max) for a base value R.
  - ⑤ set of physical address corresponding to logical address is known as physical address space.

मेहनत करना पड़ता है कुछ पाने के लिए आराम ही करो
अच्छे ही करो

What is paging (non contiguous memory allocation)

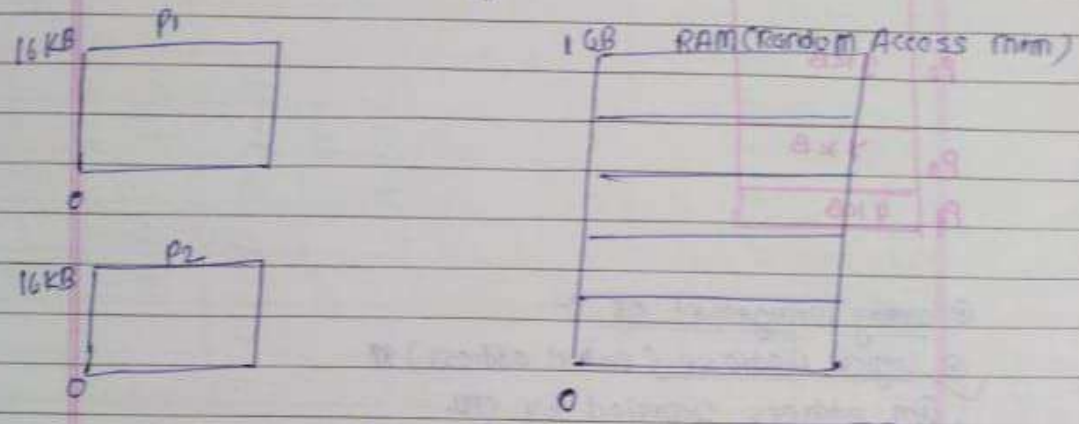


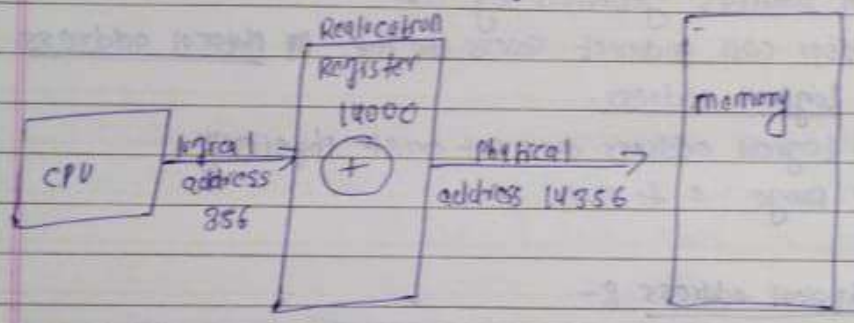
- memory management as :-
- Logical Address (virtual address)
  1. a address generated by cpu.
  2. User can indirect access the the physical address through logical address.
  3. Logical address does not exist physically.
  4. Range : 0 to max.

- Physical address :-
  1. User can never access physical address of the programe.
  2. a physical address can be accessed by a user indirectly but not directly.
  3. it's computed by (mmu) (memory management unit)
  4. Range: (R+0) to (R+ max) for a base value R.
  5. set of physical address corresponding to logical address is known as physical address space.

- Runtime mapping from virtual to physical address is done by hardware device called memory management unit.

- the user programe mainly





User think that programe running at logical address. but the programe mainly needs physical memory in order to complate the Execution.

- How os Manage Isolation and protection?

Lec :- 24 memory-management techique

Logical versus Physical address space

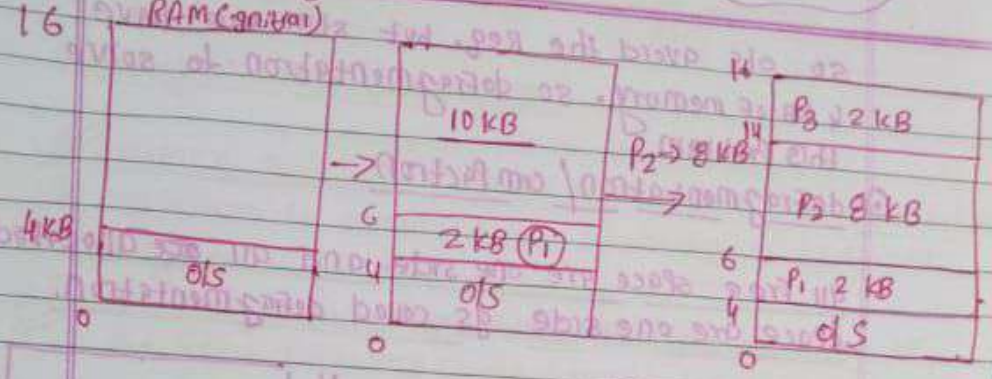
(a) Logical address
-> Logical address genrated by CPU.
-> User can access Logical address of Process.
-> User access Physical address through Logical address.
-> set of all logical address that are genrated by any Programe is called Logical address space.
-> Range 0 to max.

(b) Physical address
-> User never access Physical address of the Programe.
-> Physical address can be accessed by user indirectly but not directly.
-> set of all physical address corresponding to the logical address is known as Physical address space
-> it's computed by memory management unit
-> Range (R+0) to (R+max) for base value R

(*) The runtime mapping from logical (virtual) to Physical address is done by H/W device called the MMU (memory management unit).

(*) How OS manages the isolation and Protect?
(memory mapping and Protection)

① How OS manages free space ?



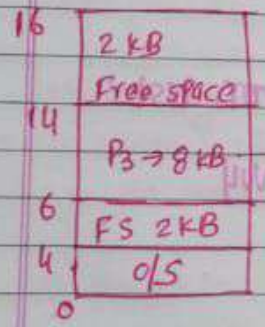
① initial Fragmentation list :-

~~4 KB~~ → 12 KB → NULL

② 10 KB → NULL

③ 2 KB → NULL

After P1 and P2 exit



~~Free list~~ Fragmentation list

2 KB → 2KB → NULL

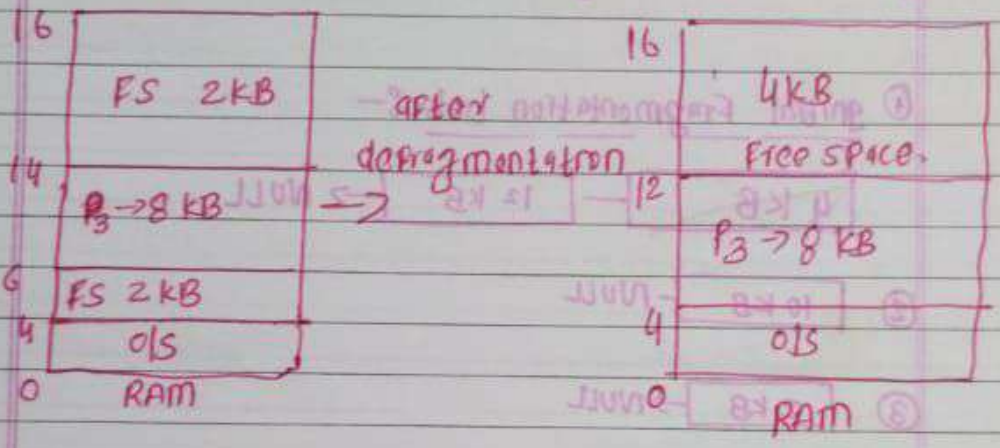
② what is the Problem ?

suppose we want to allocate P4 which is of 3 KB. O/S check fragmentation list 3 KB is not available.

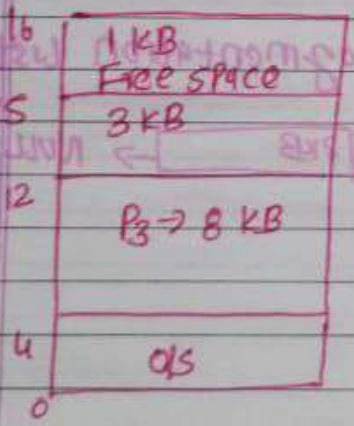
so o/s avoid the Req. but still we have 4 KB of memory. so defragmentation to solve this Problem.

⊙ defragmentation / compaction

all free space are one side and all allocated space are one side is called defragmentation.



⊙ now we can allocate 3KB of memory.



Free ~~fragmentation~~ list

1 KB	NULL

① what is paging / non contiguous memory allocation

② defragmentation / compaction

ⓐ dynamic partitioning suffer from external fragmentation
ⓑ compaction to minimize the probability of external fragmentation.
ⓒ all free partitions are made contiguous and all the loded partitions are brought together.

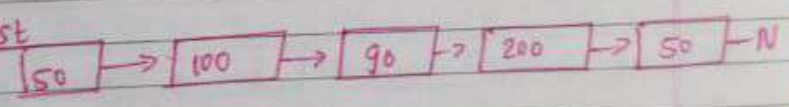
ⓓ by applying this technique we can store the bigger process in the memory. the free partition are merged which can now be allocated according to the needs of new process. this technique is called defragmentation

② How free space is stored in OS ?
Free hole in the memory are represented by Free list.

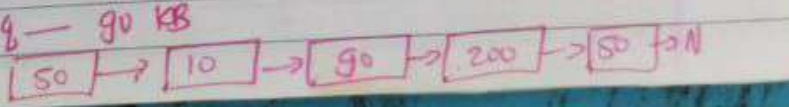
③ How to satisfy a Request of a n size from a list of free holes ?
Various algorithms which are implemented by the O/S. in order to find out the holes in the linked and allocated to the process.

① First fit :-

Free list



Req — 90 KB



from free list

(i) allocate the first hole that is big enough.
(2) simple and easy to implement
(3) Fast/less time complexity.

⊙ Next Fit
(i) Enhan cement on First Fit but start search always from last allocated hole

⊙ Best Fit
from free list
(i) allocate smallest hole that is big enough.
(ii) Lesser internal fragmentation
(iii) may create many small holes and cause major external fragmentation.

(iv) slow as required to iterate whole free holes list

⊙ Worst Fit
(i) allocate the largest hole that is big enough
(ii) slow, as required to iterate whole free hole list
(iii) Leave larger hole that may accomodate other process.

What is Paging / Non-contiguous memory allocation

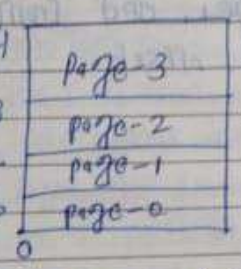
① The main disadvantages of dynamic partitioning is external fragmentation. This can be removed by compaction, but with overhead.

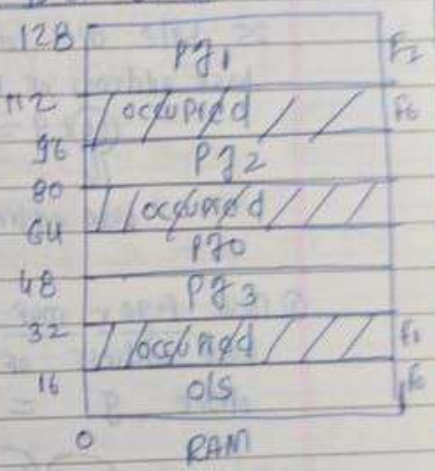
② Idea behind Paging

Paging :-

- It avoids external fragmentation and need of compaction.
- Idea is to divide the physical memory into fixed size block called frames along with divide logical memory into block of same size called pages.
- Page size = frame size
- Page size usually determine by the processor architecture traditionally page in a system has uniform size such as 4096 bytes.

③ Suppose P1 is a process which is of 64 kB.





Page = logical address
Fixed size division
and Page size = Frame size

Physical memory divided into fixed partition is called Frame

- each process should have its own page table.
- page table

Logical page-no	Physical frame no
page-0 →	Frame-3
page-1 →	Frame-7
page-2 →	Frame-5
page-3 →	Frame-2

- How address translation done.

suppose we want to access 25 position.

25 = 01 1001

page no offset

25 byte are persented at page1, and from base address of this page to add offset

16 + 9 ⇒ offset

base address

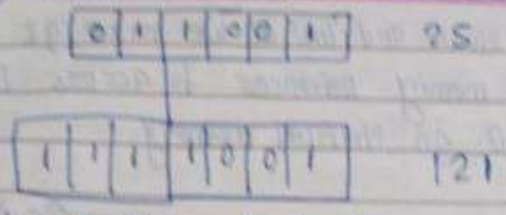
- now page1 map with frame-7
base address of frame-7 is 112. and add offset 9 = 121

121 = 111 1001

frame no d

Frame no = 7

Logical address space



Physical address space

first 25 = 01 1001. first two bit are page no here 01 means page1 and page1 map with frame 7 from page table. and base address of frame 7 is 112. add offset 9 than 121 is a address of page1 in physical memory.

Page table

- a data structure store which page is mapped to which page.
- the page table contain the base address of each page in the physical memory.
- Every address generated by CPU (logical address) is divided into two parts, a page no (P) and page offset (d) the P is used as an index into a page table to get base address the corresponding frame in physical memory.
- page table is stored in main memory at the time of process creation. and it's base address is stored in PCB (process control block).

How paging avoids external fragmentation :-

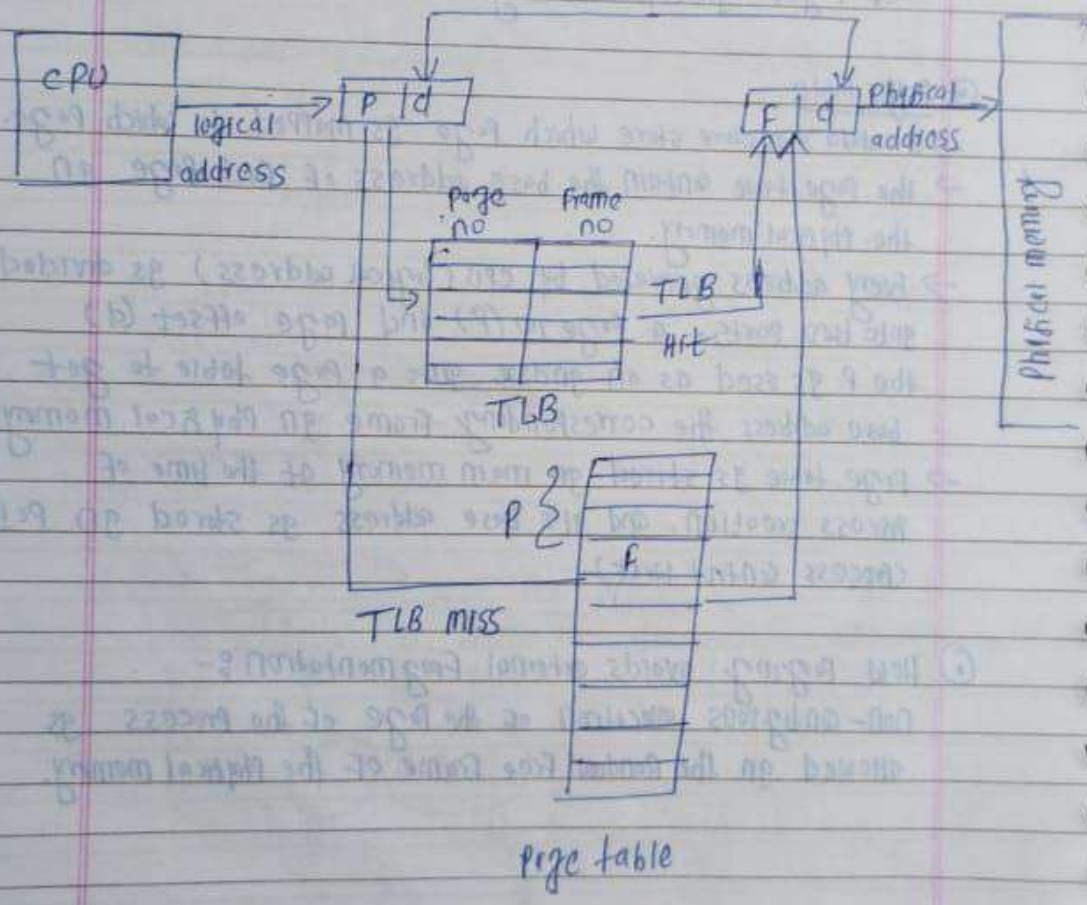
Non-contiguous allocation of the page of the process is allowed in the random free frame of the physical memory.

⊙ why Paging is slow. and how do we make it fast?

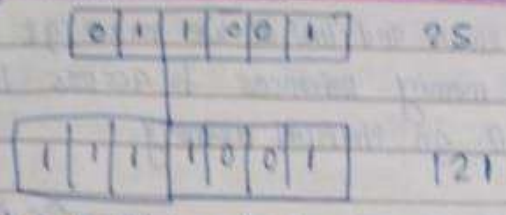
⇒ there are too many memory refrances to access the desired location in physical memory.

Logical address से physical address पर जाने के लिए हमें Ptable load करना होता है। इसलिए slow है।

⊙ Translation Look-aside buffer (TLB)



Logical address space



Physical address space

first 25 = 01 1001. first two bit are page no here 01 means page1 and page1 map with frame 7 from page table. and base address of frame 7 is 112. add offset 9 than 121 is a address of page1 in physical memory.

Page table

- a data structure store which page is mapped to which page.
- the page table contain the base address of each page in the physical memory.
- Every address generated by CPU (logical address) is divided into two parts, a page no (P) and page offset (d) the P is used as an index into a page table to get base address the corresponding frame in physical memory.
- page table is stored in main memory at the time of process creation. and it's base address is stored in PCB (process control block).

How paging avoids external fragmentation :-

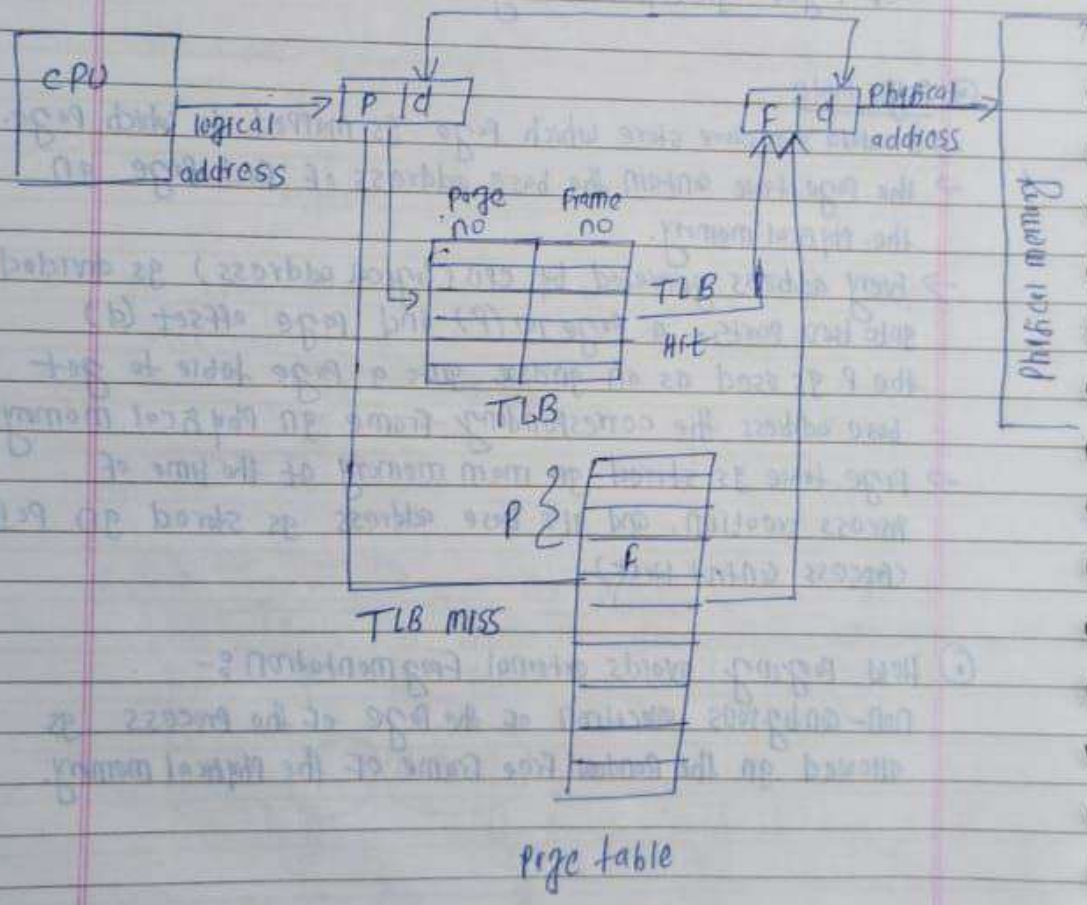
Non-contiguous allocation of the page of the process is allowed in the random free frame of the physical memory.

⊙ why Paging is slow. and how do we make it fast?

⇒ there are too many memory refrances to access the desired location in physical memory.

Logical address से physical address पर जाने के लिए हमें Ptable load करना होता है। इसलिए slow है।

⊙ Translation Look-aside buffer (TLB)



CPU Generate logical address, contain p and d
p → page no and d = offset. First page Entry check in TLB if found then it's not go to page table. if not found then go to page table and find frame. corresponding to page and this Entry push in TLB. ~~and add d to it to find~~ and find base address of this frame. add d to it. to find particular page in physical memory.

if page found in TLB than it's called TLB hit. else it's called TLB miss.

Q when context switching done, than new process came and for each process a different page table. but in TLB still have information related to previous process. so first solution is to delete all entry from TLB when context switching happen. but it's costly. so make new TLB like this

→ Address space identifier

ASID	Pg no	Frame no
0	10	100
1	10	45

now suppose we search pg no-10 and pg-no 10 found. but check it belong to given process or not. if it belong to given process than TLB will be hit. else TLB miss.

Lec 2-27

what is the problem with paging :-

suppose physical memory frame size 2KB

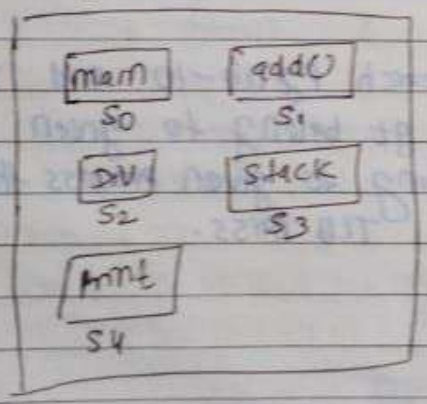
and suppose any process fun() size is 4KB

and suppose 2KB store in F_1 and 2KB store in F_4.

so cpu execute half instruction from F_1 and context switch to F_4. so this is overhead.

segmentation solve this problem

suppose we have any programe. which consist of following function.



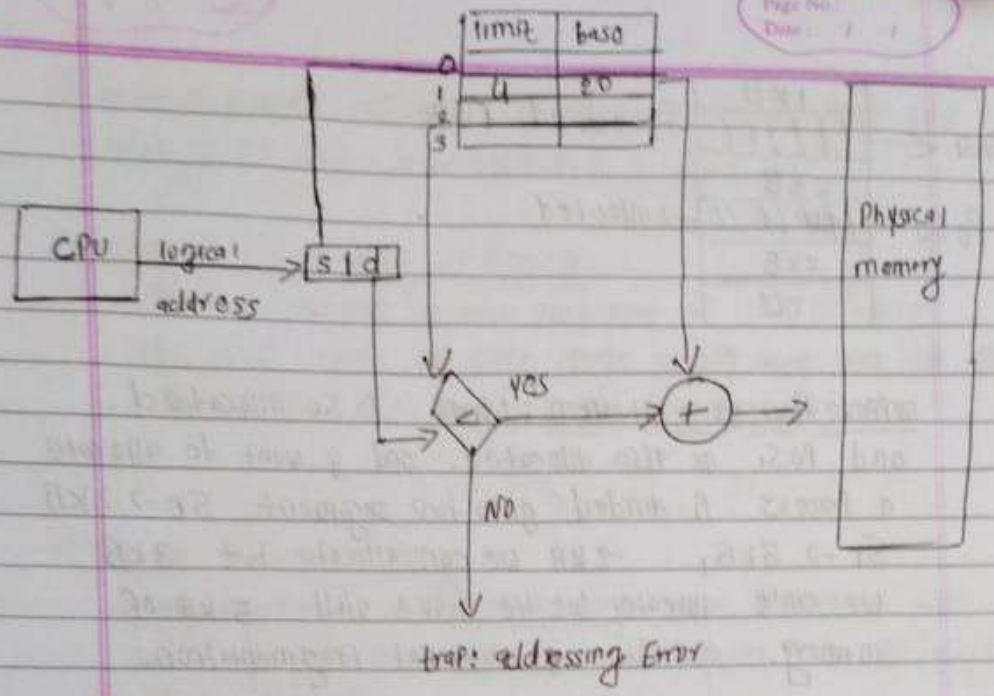
all segment size is different

$S_0 \rightarrow 4KB$

$S_1 \rightarrow 8KB$

$S_3 \rightarrow 10KB$

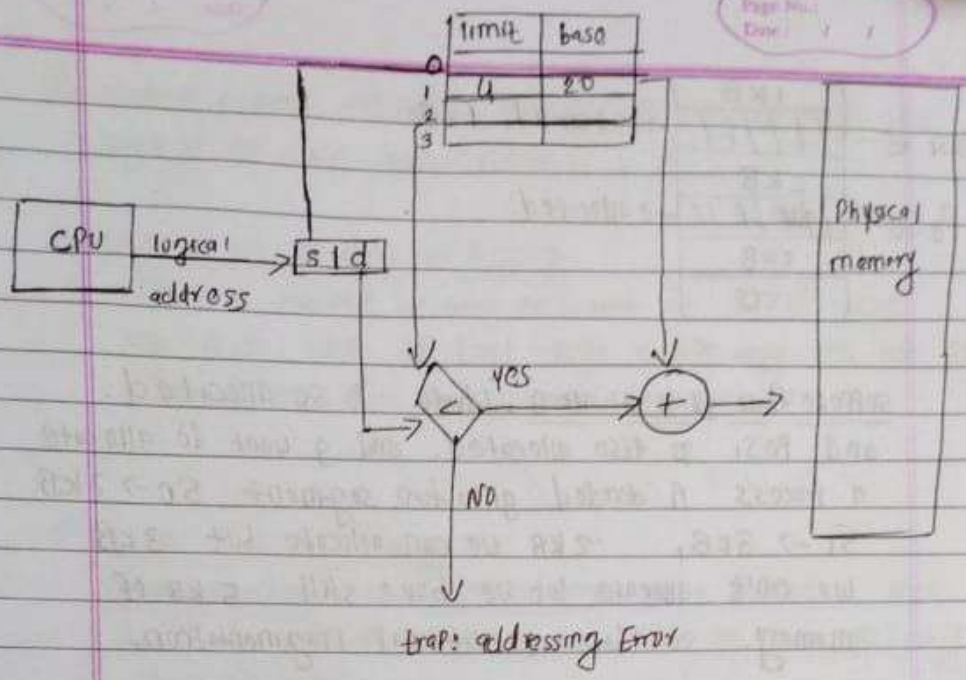
and so on



- MMU (memory management unit) convert logical address into physical address.

- How address translation done. $\boxed{LA \rightarrow P.A}$
  suppose logical address is 01 01 1 it see at index 1
  (01 → s, 01 1 → d)
  segment 1 have limit 4 and it's base address is 20 in physical memory.
  check limit < d or not if limit < d than fetch 20 + d from physical memory.

- external fragmentation in segmentation :-



① MMU (memory management unit) convert logical address into physical address.

② How address translation done. $\boxed{LA \rightarrow P.A}$

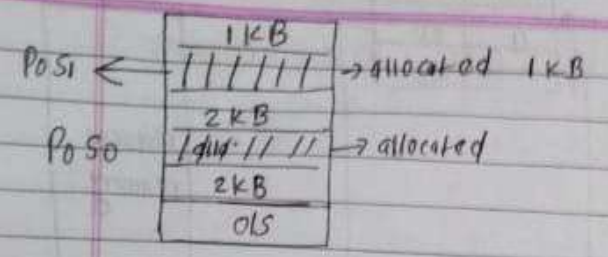
suppose logical address is (01)(01 1) it see at index 1

s d

segment1 have limit 4 and it's base address is 20 in physical memory.

check limit < d or not if limit < d than fetch 20 + d from physical memory.

③ external fragmentation in segmentation :-



suppose than is a situation. where P0 S0 allocated. and P0S1 is also allocated. and I want to allocate a process P divided into two segment S0 → 2KB S1 → 3KB, 2KB we can allocate but 3KB we can't allocate but we have still 5KB of memory. so there is external fragmentation.

① Advantages

(1) No internal fragmentation.
(2) size of segment table is less than size of page table.
(3) It's more efficient system because the compiler keeps the same type of function in one segment

② disadvantages

(a) external fragmentation
(b) the different size of segment is not good that the time swapping.
जो segment ज्यादा use होता है उसे CPU High Rank देती है और जो कम use होता है उसे low Rank. तो low Rank segment swapout हो जाता है और अलग - अलग size का segment swap-in, swap-out होगा जो efficient नहीं होगा।

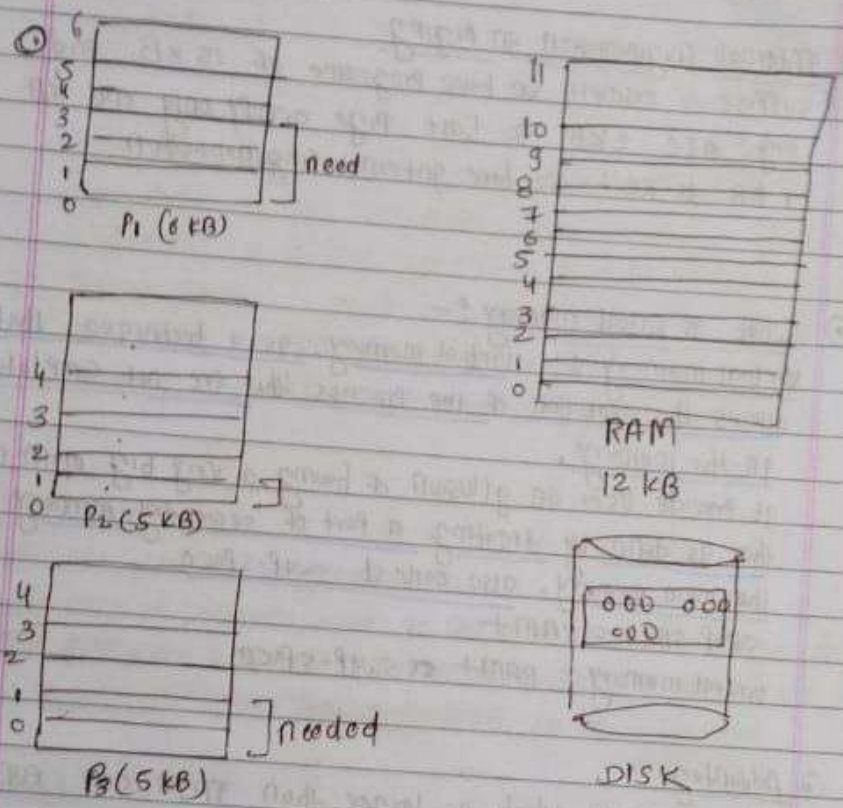
(a) modern system architecture provide both segmentation and paging in some hybrid approach.

(b) Internal fragmentation in paging
suppose a example we have programe of 15 KB. and page size 2KB so last page occupy only one KB in ram 1 KB is rest. so here internal fragmentation

(c) What is virtual memory :-
Virtual memory :- Virtual memory is a technique that allows the execution of the process that are not completely in the memory.
it provide user an illusion of having a very big main memory. this is done by treating a part of secondary memory as the main memory. also called swap-space.
swap-space = RAM +
virtual memory = RAM + swap-space

(d) Advantages
→ the programe which is larger than RAM can run.
→ Programmer is provided very large virtual memory when only a smaller physical memory is available.
→ demand paging is a popular method of virtual memory management.
→ in demand paging, the pages of a process which are least used, get stored in the secondary memory.
→ a page is copied to the main memory when its demand is made or page fault occurs

there are various page-Replacement algorithm. which are Used to determine the page which will be Replaced.



- P1 is of 6kB but only 3 kB are needed.
- P2 is of 5kB but only 2kB are needed.
- P3 is of 5 kB but only 2 kB are needed.

so only needed part copy in the memory. Rest of the part swap-out in the secondary memory and when needed than again swap-in in main memory. 1 GB are Reserved in disk

~~For~~ and this space is called swap-space
and virtual memory = RAM space + swap-space.

→ User's illusion that all programe run in RAM
→ large process, which have larger than the size of RAM can run.

⑥ demand paging

→ Rather than swapping the entire process into memory. we use Lazy swapper. A lazy swapper never swaps a page into memory unless that page will be needed.

→ We are viewing a process as a sequence of pages. swapper wpy entire process in the RAM, pager wpy individual pages of process.

⑦ How demand paging works?
ⓐ when a process is to be swapped-in, the pager guess which pages will be used.
ⓑ the page which is not used, not loded in the RAM. that's way swap time decrease.
ⓒ the valid-invalid bit scheme in the page table. is used to distinguish b/w pages that in the memory and that are on the disk.

① valid-invalid bit 1 means the associated page is both legal and in memory.
② valid invalid bit 0 means page either not valid or is valid but currently on the disk

frame
valid - invalid bit
A
B
C
D
E
F
G
H
4
6
9
A
C
E
D
E
G
H
physical memory
page table
physical memory
user space